Wildcat4626Wrapper Audit

Côme du Crest

2025-12-21

Table of contents

- Table of contents
- Wildcat4626Wrapper Audit
  - Scope
  - Context
  - Status
  - Legal Information And Disclaimer
- Issues
  - [Info] Incorrect rounding of maxWithdraw
  - [Info] Incorrect rounding of preview functions
  - [Info] Sanctioned caller of max functions

Wildcat4626Wrapper Audit

This document presents the findings of a smart contract audit conducted by Côme du Crest.

For any inquiry contact me by email at come.ducrest@gmail.com or via telegram @ComeduCrest.

Scope

The scope includes all contracts within wildcat-finance/v2-protocol/src/vault as of commit `0xf8d8b9b` currently on branch `feat-4626-wrapper`.

Context

Wildcat markets are non-collateralized lending markets where market owners borrow tokens from lenders for a continuously applied borrowing rate. The market contracts use a `scaleFactor` (in `ray`) to represent the increasing amount of interests accrued by the markets. The functions exposed by the market use `normalized` balances that are increasing over time as the `scaleFactor` increases. I.e. the function `balanceOf(user)` will return an increasing amount over time. Internally, the contract uses `scaled` balances that are constant over time for accounting such as transfer computations.

The balances follow the formulas: `normalized = scaled * scaleFactor / 1e27` and `scaled = normalized * 1e27 / scaleFactor`.

When a user calls `transfer(to, amount)`, the amount is specified in `normalized` unit and needs to be converted to `scaled` amount which involves a division. The contract uses half-up rounding for that division.

To enhance composability an ERC4626 wrapper has been implemented that expose the constant over time `scaled` balances. The `Wildcat4626Wrapper` allows holders of `WildcatMarket` tokens to mint vault shares for which the `balanceOf()` function will return constant values over time and for which the `transfer()` function has a straightforward behaviour. The amount of market tokens that can be redeemed for each share increases over time as the underlying market `scaleFactor` increases. This is an expected behaviour of vault tokens.

The wrapper needs to account for the half-up rounding of the transfer function of the underlying market.

On rounding of mint()

The `mint()` function used to mint an exact amount of shares in exchange for assets computes the amount of underlying assets to be transferred, and recomputes the amount of shares that would result

from this transfer to assert the expected amount of shares is equal to the user requested amount of shares. Due to rounding occurring at multiple steps of the calculation, I initially believed the assertion could revert under certain circumstances. After careful analysis the calculation is correct. Here are my notes.

The function calculates the amount of assets the user needs to deposit to mint the exact amount of `shares` with:

$$assets = \frac{shares * scaleFactor - \frac{scaleFactor}{2} + RAY - 1}{RAY} \tag{1}$$

It then computes the expected number of shares that would be received if `assets` is deposited via:

$$expectedShares = _convertToSharesHalfUp(assets, scaleFactor) \tag{2}$$

$$expectedShares = \frac{assets * RAY + \frac{scaleFactor}{2}}{scaleFactor} \tag{3}$$

$$expectedShares = \frac{\frac{shares*scaleFactor - \frac{scaleFactor}{2} + RAY - 1}{RAY} * RAY + \frac{scaleFactor}{2}}{scaleFactor} \tag{4}$$

The division and multiplication by `RAY` results in ceiling rounding of `shares*scaleFactor - scaleFactor/2` which may lead to an off by one error in the `expectedShares` calculation. In this case, the expected share will be:

$$expectedShares = \frac{shares * scaleFactor + RAY}{scaleFactor} \tag{5}$$

$$expectedShares = shares + \frac{RAY}{scaleFactor} \tag{6}$$

Which means expectedShares is equal to shares if `scaleFactor > RAY` and the rounding is never an issue. The contract can then enforces `expectedShares == shares`.

```
function mint(
  uint256 shares,
  address receiver
) public override nonReentrant returns (uint256 assets) {
  ...
  uint256 scaleFactor = wrappedMarket.scaleFactor();
  ...

  //  minimum assets for half-up rounding to yield `shares`
  uint256 numerator = shares * scaleFactor;
  uint256 halfSf = scaleFactor / 2;
  if (numerator <= halfSf) revert ZeroAssets();
```

```
13      assets = (numerator - halfSf + RAY - 1) / RAY; // ceiling
14
15      // Verify the formula produced the correct result
16      uint256 expectedShares = _convertToSharesHalfUp(assets, scaleFactor);
17      if (expectedShares != shares) revert SharesMismatch(shares, expectedShares)
            ;
18
19      ...
20    }
21
22    function _convertToSharesHalfUp(
23      uint256 assets,
24      uint256 scaleFactor
25    ) internal pure returns (uint256) {
26      return (assets * RAY + scaleFactor / 2) / scaleFactor;
27    }
```

Status

The report has been sent to the core developer.

No serious issues have been found, only `info` level issues have been reported.

Legal Information And Disclaimer

1. This report is based solely on the information provided by Wildcat Protocol (the “Company”), with the assumption that the information provided is authentic, accurate, complete, and not misleading as of the date of this report. The auditor has not conducted any independent enquiries, investigations or due diligence in respect of the Company, its business or its operations.
2. Changes to the information contained in the documents, repositories and any other materials referenced in this report might affect or change the analysis and conclusions presented. The auditor is not responsible for monitoring, nor will we be aware of, any future additions, modifications, or deletions to the audited code. As such, the auditor does not assume any responsibility to update any information, content or data contained in this report following the date of its publication.
3. This report does not address, nor should it be interpreted as addressing, any regulatory, tax or legal matters, including but not limited to: tax treatment, tax consequences, levies, duties, data privacy, data protection laws, issues relating to the licensing of information technology, intellectual property, money laundering and countering the financing of terrorism, or any other legal restrictions or prohibitions. The auditor disclaims any liability for omissions or errors in the findings or conclusions presented in this report.
4. The views expressed in this report are solely my views regarding the specific issues discussed within this report. This report is not intended to be exhaustive, nor should it be construed as an assurance, guarantee or warranty that the code is free from bugs, vulnerabilities, defects or deficiencies. Different use cases may carry different risks, and integration with third-party applications may introduce additional risks.
5. This report is provided for informational purposes only and should not be used as the basis for making investment or financial decisions. This report does not constitute investment research and should not be viewed as an invitation, recommendation, solicitation or offer to subscribe for or purchase any securities, investments, products or services. The auditor is not a financial advisor, and this report does not constitute financial or investment advice.
6. The statements in this report should be considered as a whole, and no individual statement should be extracted or referenced independently.
7. To the fullest extent permitted by applicable laws, the auditor disclaims any and all other liability, whether in contract, tort, or otherwise, that may arise from this report or the use thereof.

Issues

[Info] Incorrect rounding of maxWithdraw()

Summary

The function `maxWithdraw()` uses incorrect rounding and an additional asset could be withdrawn by the user.

Vulnerability Detail

The function `maxWithdraw()` uses half-up rounding on the amount of shares held by the user to return the max amount of withdrawable assets:

```
function maxWithdraw(address owner_) public view override returns (uint256) {
  if (_isSanctioned(owner_)) return 0;
  uint256 shares = balanceOf(owner_);
  if (shares == 0) return 0;
  uint256 scaleFactor = wrappedMarket.scaleFactor();
  return _convertToAssetsHalfUp(shares, scaleFactor);
}

function _convertToAssetsHalfUp(
  uint256 shares,
  uint256 scaleFactor
) internal pure returns (uint256) {
  return (shares * scaleFactor + RAY / 2) / RAY;
}
```

The function `withdraw()` also uses half-up rounding to get the amount of shares to burn from the user to withdraw assets:

```
function withdraw(
  uint256 assets,
  address receiver,
  address owner_
) public override nonReentrant returns (uint256 shares) {
  ...

  uint256 scaleFactor = wrappedMarket.scaleFactor();
  shares = _convertToSharesHalfUp(assets, scaleFactor);
  ...

  uint256 scaledBefore = wrappedMarket.scaledBalanceOf(address(this));

  _burn(owner_, shares);
  address assetAddress = address(wrappedMarket);
  assetAddress.safeTransfer(receiver, assets);
  uint256 scaledAfter = wrappedMarket.scaledBalanceOf(address(this));
```

```
18
19      uint256 burnedShares = scaledBefore - scaledAfter;
20      if (burnedShares != shares) revert SharesMismatch(shares, burnedShares);
21      ...
22    }
23
24    function _convertToSharesHalfUp(
25      uint256 assets,
26      uint256 scaleFactor
27    ) internal pure returns (uint256) {
28      return (assets * RAY + scaleFactor / 2) / scaleFactor;
29    }
```

However, the two roundings do not cancel each other as they have different cutoffs and an off by one error could occur.

Impact

User may be able to withdraw more assets than what is returned by `maxWithdraw()` by burning the same amount of shares.

Code Snippets

Here is a test as an example found through fuzzing that can be added to `Wildcat4626WrapperRounding.t.sol` showing that `maxWithdraw + 1` can be used to withdraw from the wrapper:

```
1     function test_maxWithdraw() external {
2       uint depositAssets = 27844959902324867149414625;
3       uint scaleOffset = 162749548383032133084497237495292590326 9;
4       market.setScaleFactor(RAY + scaleOffset);
5
6       vm.prank(ALICE);
7       uint256 depositedShares = wrapper.deposit(depositAssets, ALICE);
8
9       uint256 maxWithdraw = wrapper.maxWithdraw(ALICE);
10
11      uint snapshot = vm.snapshot();
12      vm.prank(ALICE);
13      uint256 sharesBurned = wrapper.withdraw(maxWithdraw, ALICE, ALICE);
14      vm.revertToAndDelete(snapshot);
15      vm.prank(ALICE);
16      vm.expectRevert();
17      sharesBurned = wrapper.withdraw(maxWithdraw + 1, ALICE, ALICE);
18    }
```

https://github.com/wildcat-finance/v2-protocol/blob/f8d8b9babbe02e5c4d4294abbd8661bb7cf57c10/src/vault/Wildcat4626Wrapper.sol#L170-L176

Recommendation

The issue can be acknowledged. Alternatively, review the calculation to perform the inverse operation of `withdraw()` in `maxWithdraw()`.

[Info] Incorrect rounding of preview functions

Summary

The preview functions `previewDeposit()`, `previewMint()`, `previewWithdraw()`, and `previewRedeem()` all round in favour of protocol (conservative rounding) and do not replicate the rounding of corresponding `deposit()`, `mint()`, `withdraw()`, and `redeem()` function.

Vulnerability Detail

The function `previewDeposit()` simply converts the deposited assets to shares and return this value:

```
function previewDeposit(uint256 assets) public view override returns (uint256
    ) {
  return convertToShares(assets);
}
```

However, the `deposit()` function uses half-up rounding for the amount of shares received by the receiver:

```
function deposit(
  uint256 assets,
  address receiver
) public override nonReentrant returns (uint256 shares) {
  ...
  uint256 scaleFactor = wrappedMarket.scaleFactor();
  uint256 expectedShares = _convertToSharesHalfUp(assets, scaleFactor);
  if (expectedShares == 0) revert ZeroShares();

  address assetAddress = address(wrappedMarket);
  uint256 scaledBefore = wrappedMarket.scaledBalanceOf(address(this));
  assetAddress.safeTransferFrom(msg.sender, address(this), assets);
  uint256 scaledAfter = wrappedMarket.scaledBalanceOf(address(this));

  shares = scaledAfter - scaledBefore;
  if (shares < expectedShares) revert SharesMismatch(expectedShares, shares);

  _mint(receiver, shares);
  ...
}
```

The EIP for the `previewDeposit()` function states:

> MUST return as close to and no more than the exact amount of Vault shares that would be minted in a deposit call in the same transaction.

Due to different rounding the preview function does not return `as close to` the exact amount of Vault shares that would be minted in the corresponding deposit.

The behaviour of the three other preview and effect function is analogous.

Impact

Preview functions may be off by one due to rounding compared to their respective effect functions which is not in strict adequation to the EIP.

Code Snippets

https://eips.ethereum.org/EIPS/eip-4626

https://github.com/wildcat-finance/v2-protocol/blob/f8d8b9babbe02e5c4d4294abbd8661bb7cf57c10/src/vault/Wildcat4626Wrapper.sol#L148-L150

Recommendation

Use the same rounding in preview and effect functions.

[Info] Sanctioned caller of max functions

Summary

The functions to get the max deposable, mintable, withdrawable, or redeemable amounts do not check whether `msg.sender` is sanctioned and will be able to interact at all. Which may be unexpected for ERC4626.

Vulnerability Detail

The function `maxDeposit()` checks whether `receiver` is sanctioned but not `msg.sender`:

```
function maxDeposit(address receiver) public view override returns (uint256)
    {
  if (_isSanctioned(receiver)) return 0;
  uint256 marketCap = wrappedMarket.maxTotalSupply();
  uint256 held = totalAssets();
  if (held >= marketCap) return 0;
  return marketCap - held;
}
```

However, the `deposit()` function checks that `msg.sender` is not sanctioned before proceeding with the deposit:

```
function deposit(
  uint256 assets,
  address receiver
) public override nonReentrant returns (uint256 shares) {
  _checkNotSanctioned(msg.sender);
  ...
}
```

The EIP states about `maxDeposit()`:

> Maximum amount of the underlying asset that can be deposited into the Vault for the receiver, through a deposit call.
>
> MUST return the maximum amount of assets deposit would allow to be deposited for receiver and not cause a revert, which MUST NOT be higher than the actual maximum that would be accepted (it should underestimate if necessary). This assumes that the user has infinite assets, i.e. MUST NOT rely on balanceOf of asset.
>
> MUST factor in both global and user-specific limits, like if deposits are entirely disabled (even temporarily) it MUST return 0.

I interpret the `user` as being the caller of both the `maxDeposit()` and eventual `deposit()` functions. As such, the `maxDeposit()` should return 0 if `msg.sender` is not allowed to deposit.

The issue is analogous for the `maxMint()`, `maxWithdraw()`, and `maxRedeem()` functions.

Impact

Discrepancy with ERC4626 for `maxDeposit()`, `maxMint()`, `maxWithdraw()`, and `maxRedeem()` which may return non-zero values for a user that may fail to call the corresponding action functions due to being sanctioned.

Code Snippets

https://github.com/wildcat-finance/v2-protocol/blob/f8d8b9babbe02e5c4d4294abbd8661bb7cf57c10/src/vault/Wildcat4626Wrapper.sol#L139-L145

https://eips.ethereum.org/EIPS/eip-4626

Recommendation

Check that `msg.sender` is not sanctioned in max functions.

Alternatively, acknowledge this issue if you interpret the ERC differently and this is a bother for some reason.